

Experiment-07: Breadth-First Search (BFS) using Python

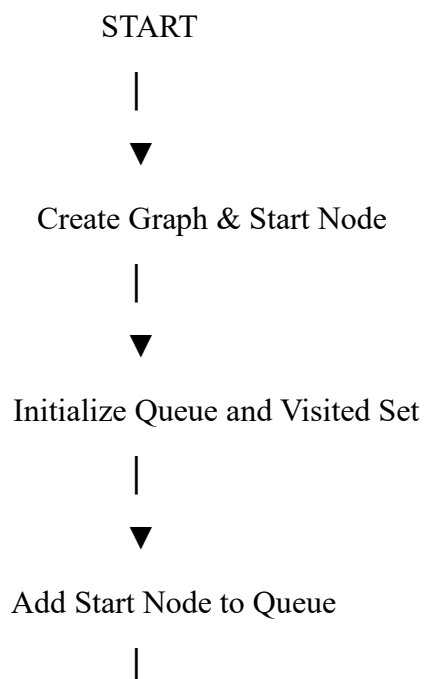
Aim

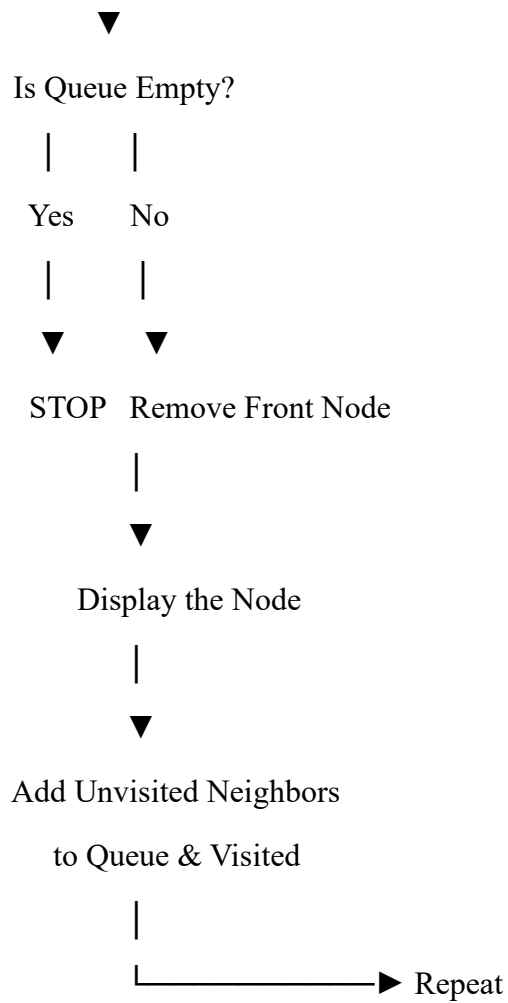
To develop a Python program to implement the **Breadth-First Search (BFS)** algorithm for traversing a graph and visiting all reachable nodes level by level.

Algorithm

1. Create the graph using an adjacency list.
 2. Choose the starting node.
 3. Create an empty queue and a visited set.
 4. Insert the starting node into the queue and mark it as visited.
 5. Remove the front node from the queue.
 6. Display the node.
 7. Visit all unvisited adjacent nodes, mark them as visited, and add them to the queue.
 8. Repeat Steps 5–7 until the queue becomes empty.
 9. Stop.
-

Flowchart





Python Code

```
from collections import deque
```

```
# BFS Function
```

```
def bfs(graph, start):
```

```
    visited = set()
```

```
    queue = deque()
```

```
    visited.add(start)
```

```
    queue.append(start)
```

```
print("BFS Traversal:")

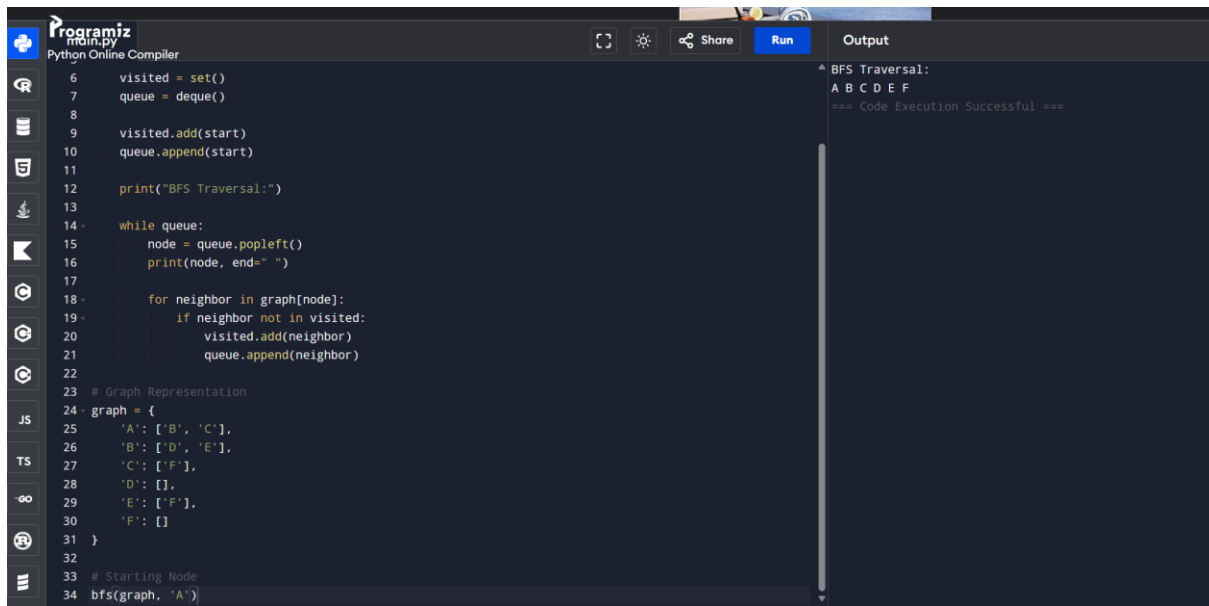
while queue:
    node = queue.popleft()
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)

# Graph Representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Starting Node
bfs(graph, 'A')
```

Output



```
6 visited = set()
7 queue = deque()
8
9 visited.add(start)
10 queue.append(start)
11
12 print("BFS Traversal:")
13
14 while queue:
15     node = queue.popleft()
16     print(node, end=" ")
17
18     for neighbor in graph[node]:
19         if neighbor not in visited:
20             visited.add(neighbor)
21             queue.append(neighbor)
22
23 # Graph Representation
24 graph = {
25     'A': ['B', 'C'],
26     'B': ['D', 'E'],
27     'C': ['F'],
28     'D': [],
29     'E': ['F'],
30     'F': []
31 }
32
33 # Starting Node
34 bfs(graph, 'A')
```

Output

```
^ BFS Traversal:
A B C D E F
=== Code Execution Successful ===
```

Result

The **Breadth-First Search (BFS)** algorithm was successfully implemented in Python. The program traversed the graph level by level using a queue and visited all reachable nodes without revisiting any node. The traversal order obtained was:

A → B → C → D → E → F